

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al. 2023) is fast and visually strong, but its efficiency comes from approximating each 3D Gaussian by a 2D projected footprint; the resulting quantitative loss is most visible for generic large field-of-view (FoV) cameras. Existing extensions improve either camera coverage or speed, yet none simultaneously match the two empirical targets that matter in practice: *projective exactness* and *real-time efficiency* for general cameras. We present 3DGEER, a geometrically exact and efficient Gaussian rendering framework designed around this comparison. Our closed-form ray integral computes Gaussian density along each ray from first principles, supporting accurate forward rendering and differentiable optimization under arbitrary camera models. For efficiency, the *Particle Bounding Frustum (PBF)* gives tight ray-Gaussian association without BVH traversal, while the *Bipolar Equiangular Projection (BEAP)* standardizes FoV representation, accelerates association, and improves supervision. Across pinhole and fisheye benchmarks, 3DGEER is consistently best across the reported quality metrics, is $5\times$ faster than existing projective-exact ray-based baselines, and retains quality when evaluated on wider FoVs than those seen during training, establishing a quantitatively favorable state-of-the-art tradeoff for real-time radiance field rendering.

1 INTRODUCTION

Since 3D Gaussian Splatting (3DGS) (Kerbl et al. 2023), splatting-based volumetric particle renderers (Yu et al. 2024a, Yan et al. 2024, Lu et al. 2024, Kheradmand et al. 2024, Charatan et al. 2024, Mallick et al. 2024) have been valued because they pair high visual fidelity with high rendering speed. The numerical advantage comes from EWA splatting (Zwicker et al. 2001), which replaces the true image footprint of a 3D Gaussian with a locally affine 2D Gaussian approximation. This approximation is often adequate for narrow field-of-view (FoV) cameras, but it becomes the dominant source of measurable error for wide-FoV inputs: nonlinear distortion makes the true projection depart from a symmetric 2D Gaussian, especially in fisheye imagery used in robotics and autonomous driving (Rashed et al. 2021, Courbon et al. 2007, Kumar et al. 2023, Zhang et al. 2016, Yogamani et al. 2019, 2024, Sekkat et al. 2022). Recent methods broaden differentiable rendering to particular fisheye camera models (Liao et al. 2024) or more general camera models (Huang et al. 2024b), but their empirical comparisons still inherit the same 2D projective-space approximation and therefore do not provide an exact closed-form rendering baseline.

A second empirical question is whether exact rendering is needed once the Gaussian count is sufficiently large. Celarek et al. (2025) reports that scaling splatting to as many as 1M Gaussians can reduce approximation error on small object-centric scenes. Our setting is quantitatively different: large scenes allocate Gaussians unevenly across objects and frustums, so brute-force scaling quickly increases memory and compute cost. Under generic distorted cameras, we further show that the measured gap persists even after substantially increasing the Gaussian count, so scaling alone does not close the quality margin (Fig. 5).

Ray-tracing and ray-marching particle renderers provide the complementary baseline (Yu et al. 2024b, Condor et al. 2025, Mai et al. 2025, Moënné-Loccoz et al. 2024, Talegaonkar et al. 2025, Gu et al. 2025). They naturally support large-FoV cameras and avoid projective approximation error, but their quantitative weakness is runtime, especially the cost of associating every ray with

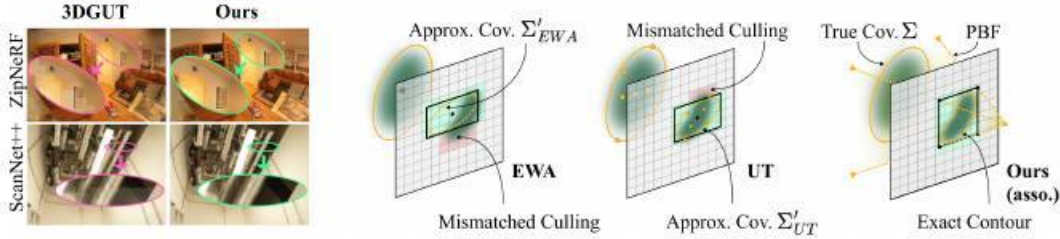


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Inaccurate UT association produces visible grid-line artifacts. (Right) The association comparison shows that 3DGEER removes the intermediate conic approximation and computes the bounding structure directly from the true 3D covariance.

the relevant 3D particles. Prior acceleration mainly relies on Bounding Volume Hierarchies (BVH), as in EVER (Mai et al. [2025]) and 3DGRT (Moënné-Loccoz et al. [2024]). BVHs improve feasibility but remain difficult to parallelize at the level needed to match 3DGS throughput. 3DGUT (Wu et al. [2025]) moves toward the splatting efficiency regime by combining ray-based rendering with rasterization-based association. The tradeoff is that association again depends on a projective approximation, which weakens geometric exactness and can introduce grid-line artifacts (Fig. 1 Left). Thus, existing comparisons separate into two groups—fast but approximate splatting methods and exact but slower ray methods—without a method occupying both favorable columns.

We propose 3DGEER, a 3D Gaussian Exact and Efficient Rendering method whose design targets the missing point in this comparison matrix: *exactness in projective geometry* (defined in Sec. A) with *high efficiency comparable to splatting* for generic cameras (both perspective and a variety of fisheye projection models). The method contributes three quantitatively motivated components: (i) We derive the Gaussian maximum-response function (Keselman & Hebert [2022]) from first principles, show why it corresponds to projective exactness, and obtain a numerically stable differentiable rendering framework. (ii) We replace costly or approximate ray-particle association with Camera Sub-frustum (CSF) to Particle Bounding Frustum (PBF) association, mirroring the tile-AABB structure of 3DGS while preserving exact geometry. For each 3D Gaussian, a closed-form PBF tightly bounds the particle, reducing unnecessary associations and making the efficiency gain explicit. (iii) We supervise colors by uniformly sampling rays in Bipolar Equiangular Projection (BEAP) space. This representation aligns image tiles with CSFs and, in the reported experiments, improves reconstruction quality relative to conventional pinhole or equidistant supervision.

Across fisheye and pinhole datasets—including ScanNet++ (Yeshwanth et al. [2023]), Zip-NeRF (Barron et al. [2023]), Aria (Lv et al. [2025]), and MipNeRF (Barron et al. [2021])—the empirical pattern is consistent: 3DGEER outperforms prior and concurrent methods, with 0.9–4.8 PSNR gains in challenging wide-FoV conditions and 0.3–2.0 gains on pinhole views. The same comparison remains favorable even when wide-FoV cameras are evaluated from models trained only on narrow-FoV data, demonstrating strong generalization to extreme-FoV rendering.

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

Most 3DGS variants (Yu et al. [2024a], Zhang et al. [2024], Rota Bulò et al. [2024], Gao et al. [2025], Zhao et al. [2025], Kerbl et al. [2024], Hou et al. [2025], Liu et al. [2025]) retain the EWA splatting strategy (Zwicker et al. [2001], Ren et al. [2002]): they project 3D Gaussians to the image plane and represent each footprint as a 2D conic. Large-FoV extensions such as FisheyeGS (Liao et al. [2024]) and GS++ (Huang et al. [2024b]) modify the 2D mean and covariance using Jacobians of equidistant or spherical projections. These changes make the methods applicable to distorted cameras, but the quantitative comparison remains bounded by a first-order Taylor approximation of a highly nonlinear projection. In wide-FoV regions, the neglected higher-order terms become large enough to reduce reconstruction metrics.

The underlying issue is geometric rather than implementation-specific: under nonlinear camera models, including both ideal pinhole and distorted projections, the exact projection of a 3D Gaussian is not a symmetric 2D Gaussian. Consequently, methods that linearize the projection inherit an

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** Methods are grouped by rendering family and compared along the attributes that determine the quality-speed tradeoff: association strategy, association scope, camera FoV support, high-FPS behavior, and whether the full method is projective-exact.

Method	Render	Particle Asso. Method	Particle Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS (Kerbl et al. 2023)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS (Liao et al. 2024)	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ (Huang et al. 2024b)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS (Huang et al. 2024a)	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF (Yu et al. 2024b)	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER (Mai et al. 2025)	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT (Moënné-Loccoz et al. 2024)	Ray	BVH	Scene	3D Gauss.	✓	✓	✓
3DGUT (Wu et al. 2025)	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS (Hahlbohm et al. 2025)	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

approximation term that can shrink in easy cases but cannot be eliminated by the 2D conic representation itself. 3DGEER removes this term by aggregating Gaussian density directly along rays in a volumetric rendering formulation, making the quality comparison reflect exact 3D contributions rather than projected approximations.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) introduces the maximum-intensity response assumption for ray-Gaussian interaction. 3DGRT (Moënné-Loccoz et al. 2024) builds on this idea to form a 3D Gaussian ray-tracing framework, enabling ray-space sampling without explicit projection and therefore reducing wide-FoV distortion error. Its quantitative weakness is speed: ray-particle association relies on costly scene-level BVH structures, which lowers frame rates relative to splatting methods.

EVER (Mai et al. 2025) takes a different route by replacing Gaussians with constant-density ellipsoids, which permits closed-form transmittance-weighted ray integration. This improves analytic tractability but changes the primitive, reducing expressiveness relative to true 3D Gaussians and retaining BVH-related runtime limits. 2DGS (Huang et al. 2024a) substitutes 3D Gaussians with 2D surfels, allowing exact ray-surface intersections. HTGS (Hahlbohm et al. 2025) reintroduces full Gaussian expressions while adopting 2DGS-style ray-splat association. Because the association remains screen-space based, its favorable regime is narrower and does not guarantee projective exactness at wider FoVs. Our contribution is to derive closed-form bounds and a full rendering pipeline in a camera-agnostic angular domain (Sec. D; Sec. D.3 details the computational relationship to 2DGS and HTGS parameterizations.

3DGUT (Wu et al. 2025), developed after 3DGRT, recovers much of the 3DGS speed profile by using splatting-style particle association. The cost of this runtime improvement is approximation in the association stage: the Unscented Transform (UT) estimates 2D conics in projective image space from finitely many sigma points. A similar limitation exists in GOF (Yu et al. 2024b), which directly applies the EWA trick to fit 2D bounding conics and projective centers for association. Under extreme-FoV conditions, imprecise AABB estimation and inaccurate tile-to-AABB associations lead to visible grid-line artifacts (Fig. 1 Left).

3DGEER instead solves ray-particle association directly at the frustum level in 3D space (Fig. 1 Right). This keeps the comparison favorable on both axes: association and rendering remain geometrically exact, while the frustum formulation preserves high rendering efficiency. Tab. 1 summarizes representative methods that aim to address projective approximation errors, highlighting key differences across several important dimensions.

3 METHOD

3DGEER is designed to make the empirical tradeoff explicit: it keeps geometric exactness while reaching real-time efficiency for arbitrary-FoV cameras. The framework addresses the two bottlenecks that separate existing baselines—*projection approximation* in splatting renderers and *BVH*

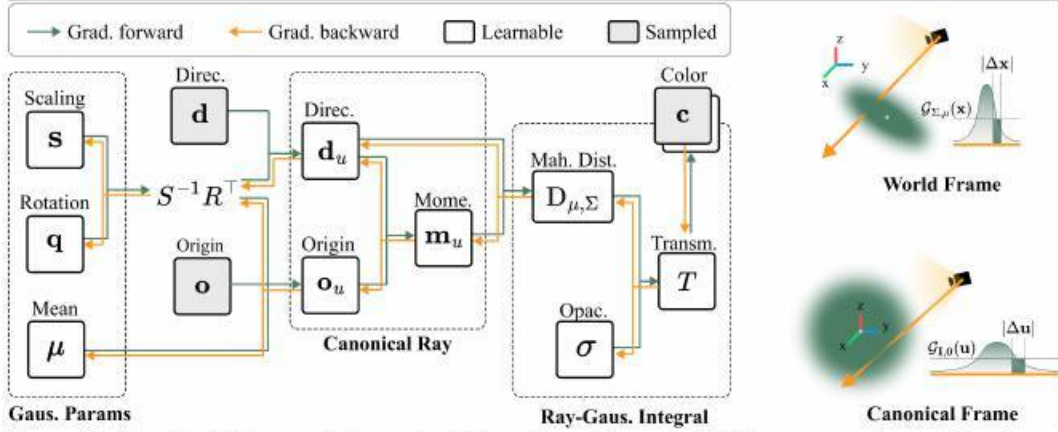


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) The compared quantities in optimization are linked through forward and backward gradients from s , q , μ , and opacity σ to transmittance. (Right) The ray integral accumulates 3D Gaussian density exactly along the ray; the green box marks the density-segment product used in Eq. B.5

overhead in ray renderers—through three components whose effects are measured in the experiments:

- Ray-based rendering formulation (Sec. 3.1): a closed-form differentiable ray integral for Gaussian density, removing the linearization error used by projection-based splatting.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a ray-particle association rule that replaces BVH traversal with exact frustum-particle tests, improving both association accuracy and association cost.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation for arbitrary FoVs that accelerates PBF computation and yields stronger reconstruction metrics under balanced supervision.

The Appendix supplies the derivational and implementation details needed to interpret these comparisons: (i) preliminaries and notation (Sec. B.1), (ii) the proof connecting maximum response to projective exactness (Sec. B), (iii) backward-gradient derivations and numerical-stability analysis (Sec. C), and (iv) the full derivation of geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

The first quantitative bottleneck is the ray integral itself: exact rendering is only practical if density and color along a ray can be evaluated in closed form. We therefore map each world-space anisotropic Gaussian to a canonical coordinate system in which the Gaussian is isotropic. Under this measure-preserving change of variables, the transmittance integral reduces to a simple closed-form expression that is efficient enough to compare directly with splatting renderers.

Canonical transformation. For each Gaussian with mean μ , rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which maps a canonical coordinate $\mathbf{u}$ to world coordinates $\mathbf{x}$. This implies $\Delta \mathbf{x} = RS \Delta \mathbf{u}$ and absorbs the Jacobian determinant into the density measure (see Fig. 2 Right). Consequently, each Gaussian reduces to the shared isotropic form

$$G_{I, 0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, its canonical form is $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ with

$$\mathbf{o}_u = S^{-1}R^T(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^T\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. The closed form shows that transmittance is determined by the ray–Gaussian distance in canonical space, so the rendering computation is both exact in projective geometry and inexpensive to evaluate. The same expression matches the “maximum response” heuristic used by prior work (Keselman & Hebert, 2022; Moënné-Loccoz et al., 2024), but our derivation identifies why it is projective-exact (Sec. B). The corresponding backward chain (Fig. 2 Left; Sec. C) also clarifies the stability advantage: it avoids the numerical failures observed in prior ray-based work (Yu et al., 2024b) and removes the need for extra post-processing such as filtering degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For ray-based Gaussian rendering, final image metrics and runtime are both controlled by the association stage, especially for large-FoV cameras where each ray can cover highly distorted angular regions. We formulate association as a CSF–PBF comparison: Camera Sub-Frustums (CSFs) are matched to Particle Bounding Frustums (PBFs), where each PBF tightly encloses a Gaussian at a prescribed distance from its center (Fig. 3). A Gaussian contributes to a CSF when the two frustums intersect, and every ray in that CSF evaluates the associated Gaussian set. This structure makes the number of candidate Gaussians per tile measurable and reducible.

To efficiently compute a PBF, we first define two spherical angles (see Fig. E.1) in the camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ denotes the normalized direction vector pointing to the camera (view) space point $\mathbf{d}_c$. The angles θ and ϕ correspond to the ray’s incidence angles in the horizontal (xz) and vertical (yz) planes, respectively, each ranging from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. We define a PBF by four planes — two aligned with the θ angle and two with the ϕ angle (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ specify the angular limits.

Given a camera with extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world transformation matrix H (See Eq. B.4), we apply the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$ to parameterize the corresponding canonical planes using the angular limits defined in view space. This transformation enables us to effectively derive the constraints for the PBFs. Take the bounds of θ as an example, the canonical plane is given by:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2}(VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, expressed as a column vector.

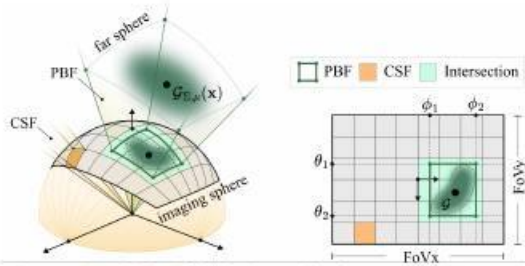


Figure 3: **PBF-CSF Association.** (Left) Each PBF uses four tangent planes to tightly bound a 3D Gaussian ellipsoid. (Right) The PBF–CSF intersection is evaluated in BEAP space, making the association count and tile coverage directly comparable across methods.

Assuming a Gaussian ellipsoid defined by a λ -standard-deviation contour, we compute the angular bounds of the local frustum by solving the following quadratic equations, which enforce that the transformed planes are tangent to the ellipsoidal contour:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ denotes the Hadamard (element-wise) product, and λ is a parameter that controls the spread of the contour, typically set to 3. An identical constraint is applied to $\mathbf{g}_v^\top$ in order to compute the corresponding angular bounds $\phi_{1,2}$. The roots of these quadratic equations determine the tangent values of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$, which define the PBF angular limits. Notably, solving these quadratic equations under canonical space constraints respects the true 3D covariance Σ_c and 3D mean μ_c of the Gaussian in the camera space. Specifically, for the bounds of θ , we have $c = \tan \theta_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation is efficient to solve (See details in Sec. D.1) and eliminates the need for intermediate conic approximations, as in EWA or UT, as well as the screen-space constraints in HTGS and 2DGS that limit their FoV.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

BEAP is introduced to make large-FoV supervision and association quantitatively cleaner. First, it represents large-FoV images without losing FoV coverage. Second, each image tile and its CSF share the same two- θ and two- ϕ parameterization, which reduces PBF association overhead and aligns naturally with GPU parallelism. Third, BEAP samples rays more uniformly in 3D than conventional projective rendering, and the ablations show that this balanced supervision improves training behavior and reconstruction metrics.

Concretely, rays are sampled uniformly in spherical angular coordinates (θ, ϕ) (Eq. 6) and then mapped to a discrete $w \times h$ supervision image by a linear scale-and-shift transform:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ indicate the camera’s horizontal and vertical field of view. Fig. 4 visualizes the resulting sampling density: uniform (θ, ϕ) sampling provides more balanced angular coverage inside the view frustum. This is the key comparison against projective sampling, which over-allocates samples to peripheral regions, and equidistant projection, which over-allocates samples near the center.

4 EXPERIMENTS

We evaluate 3DGEER under the same comparison axes used throughout the paper: camera model, dataset, training FoV, test FoV, quality metric, Gaussian count, and runtime. The benchmark suite covers both pinhole and fisheye cameras, and all default runtime numbers use an RTX 4090 GPU with 64GB of RAM unless stated otherwise. Full implementation details are given in Sec. C.

Datasets. For generic distorted-camera evaluation, where projective errors are expected to have the largest measurable effect, we use all Fisheye ZipNeRF scenes (Barron et al. 2023), five ScanNet++ scenes (Yeshwanth et al. 2023), and two Aria scenes (Lv et al. 2025). ZipNeRF and ScanNet++ provide 180° diagonal FoV inputs, while Aria provides a 110° circular FoV. For pinhole comparisons, we evaluate on Pinhole ZipNeRF and MipNeRF360 (Barron et al. 2021).

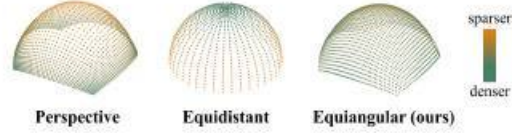


Figure 4: **Comparison of Ray Distributions under Varying Projections.** Pixels from three imaging spaces are mapped to the unit sphere so their sampling densities can be compared directly; BEAP provides the most uniform angular coverage among the shown projections.

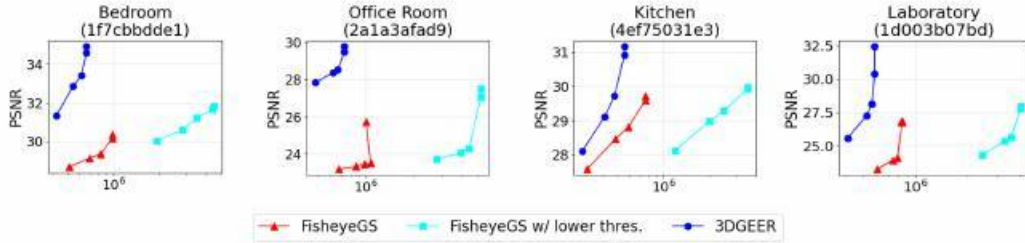


Figure 5: **PSNR $\uparrow$ Trends When Scaling-Up Gaussian Counts.** At matched or larger Gaussian counts, scaled FisheyeGS remains below 3DGEER, showing that brute-force densification does not close the projective-exactness gap (full metrics in Fig. K.3 and Tab. K.3).



Figure 6: **Extreme-FoV Generalization.** Under wider test FoVs than the training views, 3DGEER preserves complete scene structure with fewer Gaussians, whereas FisheyeGS shows severe peripheral degradation.

Evaluation Metrics and Protocol. We report PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, FPS $\uparrow$, and Gaussian count $\downarrow$, so each comparison includes both reconstruction quality and practical cost. To avoid favoring any internal representation, every result is projected back to the target image space and evaluated over the full FoV after 30k training iterations.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

We first test whether more particles can substitute for projective exactness. (Celarek et al., 2025) argues that increasing the Gaussian count to as many as 1M can reduce the linear approximation gap between splatting and ray-based rendering, but that evidence is limited to small object-centric scenes. For the wide-FoV ScanNet++ setting, we scale the splat-based fisheye baseline FisheyeGS by lowering its densification threshold, producing 3.6–5M Gaussians on an H200.

Fig. 5 compares PSNR against Gaussian count from 4k to 30k iterations, before densification typically saturates. At comparable counts, the red FisheyeGS curve remains below the blue 3DGEER curve, indicating a persistent projective-approximation margin. Even the scaled-up FisheyeGS variant saturates at 29.3 PSNR despite its much larger memory and compute footprint, whereas 3DGEER reaches 32.1 PSNR with only 550-700K Gaussians. Full statistics are reported in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We next compare 3DGEER with representative baselines on both fisheye and pinhole novel-view-synthesis benchmarks. Each dataset is paired with the evaluation condition that best reveals the relevant empirical pattern: regional wide-FoV robustness for ScanNet++, cross-camera generalization for ZipNeRF, and standard narrow-FoV quality–runtime tradeoffs for MipNeRF360.

ScanNet++. Tab. 2 separates each comparison by training FoV, supervision space, and evaluation region. Across full-FoV metrics, 3DGEER is the best method, and the regional split shows where the margin arises: baselines remain competitive near the center but lose quality in the periphery, while 3DGEER keeps high PSNR/SSIM in both regions. The qualitative comparisons in Fig. K.4

Table 2: **Comparison on the ScanNet++ Dataset.** Methods are compared by training FoV and supervision space, then evaluated on full, central, and peripheral regions to expose where wide-FoV distortion affects each baseline. Region-wise PSNR/SSIM are weighted by pixel masks; LPIPS is reported only for full FoV because it is not directly separable by region (full per-scene statistics in Tab. K.2).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Table 3: **Comparison on the ZipNeRF Dataset.** Cross-camera generalization is evaluated by training each method on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and testing on both FE and PH views, making same-camera and cross-camera margins directly comparable (full per-sequence statistics in Tab. K.9).

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
Test Data	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
Method												
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

match the numeric pattern, with sharper boundaries and fewer artifacts. Even when trained only on central crops, 3DGEER still exceeds FisheyeGS across the reported regions, showing that the gain is not limited to full-FoV supervision.

Fig. 6 further stresses generalization by training at the dataset FoV (180° diagonal FoV) and testing on a wider circular 180° FoV. Under this out-of-distribution evaluation, 3DGEER reconstructs complete scenes with fewer Gaussians, whereas splatting baselines such as FisheyeGS degrade most strongly in peripheral regions (additional extreme-FoV views in Fig. K.1 K.2).

ZipNeRF. The ZipNeRF dataset provides frame-wise aligned pinhole and fisheye views, enabling a controlled *cross-camera generalization* comparison: train on fisheye and test on pinhole, then reverse the direction. Tab. 3 shows that 3DGEER leads the methods across the reported metrics in the fisheye-trained setting and remains strongest in PSNR after pinhole training. The contrast is especially clear for cross-camera testing: FisheyeGS is strongest only in the pinhole-to-pinhole case, and 3DGUT drops sharply when a pinhole-trained model is evaluated on fisheye views. EVER and 3DGEER benefit most from geometric exactness across cameras, with 3DGEER achieving the best overall comparison. Fig. 7 shows the same pattern visually, especially in the difficult pinhole-to-fisheye condition where peripheral artifacts are most visible.

We also evaluate 3DGEER on **MipNeRF360** to verify that the gains are not restricted to fisheye cameras. Tab. 4 reports both quality and FPS: 3DGEER attains the best PSNR and LPIPS, remains competitive in SSIM, and is over $5\times$ faster than exact ray-marching/tracing baselines such as EVER and 3DGRT. It is therefore the only projective-exact method in the table with runtime close to 3DGS

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** The table reports quality and FPS together to compare accuracy–speed tradeoffs under standard pinhole views. FPS uses an RTX 4090 for our default benchmark; values sourced from original works are marked with † (RTX 6000 Ada) and ‡ (RTX 5090).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	$52^\dagger / 68^\ddagger$
3DGUT	27.26	0.810	0.218	$265^\dagger / 317^\ddagger$
3DGEER (Ours)	27.76	0.821	0.210	327

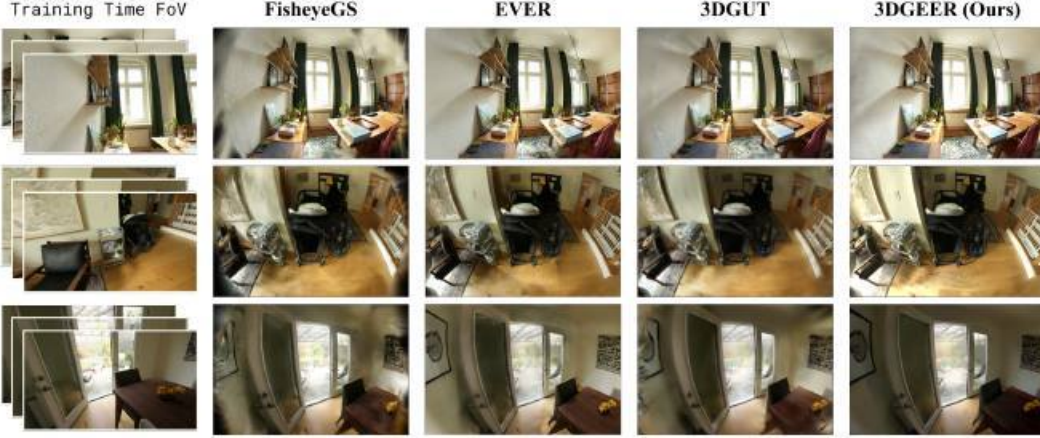


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** Visual differences are largest in the Pinhole (PH) train – Fisheye (FE) test condition, matching the quantitative cross-camera setting where 3DGEER shows its clearest advantage.

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** When the transmittance model is changed while PBF association is fixed, splatting-based transmittance uses more Gaussians but remains behind the projective-exact formulation on full-FoV quality, especially LPIPS (full per-scene statistics in Tab. K.7).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

while also improving over projective-approximation baselines (3DGS, 3DGUT) in the reported quality metrics. Additional results on **Aria**, an egocentric fisheye dataset with a moderate 110° circular FoV and low-light imaging, are provided in Sec. J.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We analyze runtime on RTX 4090 by separating total time into preparation, duplication, sorting, and rendering, then comparing 3DGEER with optimized splatting baselines (3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**). The stage-wise results in Tab. K.4 show the tradeoff clearly: ray-based rendering itself costs slightly more, but PBF substantially lowers duplication and sorting, so total runtime remains comparable to highly optimized splatting pipelines.

PBF reduces the association set to ~ 475 Gaussians per tile on average, which is $3\text{--}5\times$ fewer than EWA or UT in Tab. K.6. This reduction explains the $2.5\text{--}5\times$ association-stage speedup in Tab. K.5. Projection-based schemes such as EWA and UT overestimate screen-space extent, increasing overdraw and memory traffic; SnugBox (Hanson et al. 2025) improves their timing but does not remove the density-centering approximation that can amplify artifacts (Fig. I). The accuracy comparison is therefore aligned with the runtime comparison: PBF is faster because it is tighter, and it remains artifact-free even when ray-based rendering Gaussian fields trained with other association strategies (Tab. K.6).

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of projective-exact transmittance (Eq. 4), we keep the rest of the pipeline fixed and replace only this component with the common local-affine splat approximation, i.e., the 2D conic approximation used by 3DGS and FisheyeGS.

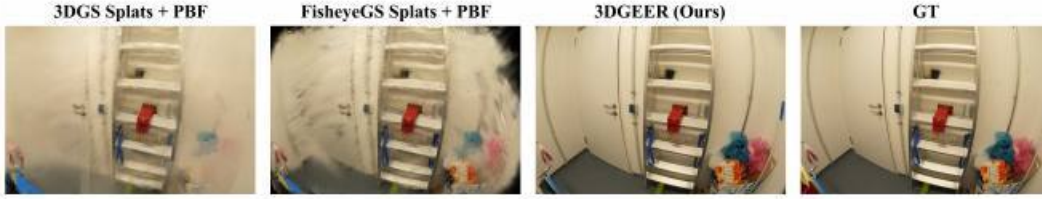


Figure 8: **Rendering-Time Transmittance Func. Replacement.** Holding training fixed and replacing projective-exact transmittance with the splatting approximation at test time isolates the mismatch: approximate transmittance with exact association leads to visibly mismatched culling.

Table 6: **BEAP vs. Alternative Supervision Schemes.** With the renderer and PBF association fixed, changing only the supervision space shows that BEAP gives the best ScanNet++ metrics across full, central, and peripheral evaluations (full statistics in Tab. K.8).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Rendering-Time Approx. At rendering time, we replace transmittance with the splatting approximation only during testing while retaining projective-exact components during training. Fig. 8 isolates the failure mode: approximate transmittance and exact association disagree about which Gaussians should contribute, producing mismatched culling. This controlled mismatch explains the gridline artifact observed in 3DGUT (Fig. I), where the inconsistency appears in the reverse direction: ray-based transmittance with conic-approximate association.

Training-Time Approx. The training-time replacement in Tab. 5 on ScanNet++ shows the same pattern quantitatively. With projective-exact association fixed, splatting-based transmittance pushes the optimizer and densification process to compensate for approximation error and cross-view inconsistency. This increases the Gaussian count by 1.6–2.4 $\times$, yet the projective-exact formulation still reports better full-FoV PSNR/SSIM/LPIPS. The remaining perceptual gap indicates that densification does not fully correct the approximation.

4.5 BEAP ON/OFF STUDY

We then isolate supervision space on ScanNet++: the projective-exact integral and PBF association are fixed, and only the ray-sampling distribution changes among pinhole, equidistant, and BEAP. Tab. 6 and Fig. K.5 show that BEAP is best across the reported metrics. The favorable pattern is consistent with its more uniform angular sampling, which supplies smoother gradients and improves convergence on fine structures.

The margin is especially informative for pinhole supervision. At fixed resolution, balanced angular information can matter more than simply exposing a larger FoV. In Tab. 6 rows 1–2 show that full-FoV perspective training can underperform central-region training, because large-FoV perspective images (Fig. K.2 Left) concentrate informative content at the center while allocating many pixels to less informative peripheral regions. BEAP improves this comparison by redistributing supervision more uniformly.

5 CONCLUSIONS

We presented 3DGEER as a closed-form, projective-exact volumetric Gaussian renderer whose empirical advantage is measured across both quality and efficiency. The PBF association module makes exact ray-particle matching practical by reducing duplication and sorting costs, and BEAP improves wide-FoV supervision by making angular sampling more balanced. Across the reported datasets and evaluation conditions, 3DGEER consistently improves the comparison against prior methods, with

540 the largest practical margins appearing in challenging wide-FoV and cross-camera settings while
541 preserving real-time performance.
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593

ETHICS STATEMENT

This work uses only publicly available research datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and does not introduce experiments involving human subjects, private data, or personally identifiable information. The quantitative evaluations are intended to advance 3D scene reconstruction and rendering for research applications such as XR, robotics, and autonomous driving. We do not identify a direct harmful consequence of the experiments reported here, but high-fidelity 3D reconstruction can raise privacy concerns if misused; accordingly, our empirical study is limited to established vision benchmarks.

REPRODUCIBILITY STATEMENT

We provide the information needed to reproduce both the method and the reported comparisons. The mathematical derivations and proofs are included in the Appendix (Secs. B, C and D); the evaluation datasets are public (ScanNet++, ZipNeRF, Aria, and MipNeRF); and preprocessing, training protocols, metrics, and evaluation conditions are specified in Sec. G. Upon publication, the implementation will be released with training and evaluation scripts so the quantitative results can be rerun.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	2
2.1 Splatting-Based Gaussian Rendering	2
2.2 Ray-Based Gaussian Rendering	3
3 Method	3
3.1 Rendering with Exact Projective Geometry	4
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	6
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	7
4.3 Comprehensive Runtime Analysis	9
4.4 Local Affine Approximation vs. Projective-Exact Integral	9
4.5 BEAP On/Off Study	10
5 Conclusions	10
a Ethics Statement	12
b Reproducibility Statement	12
c LLM Usage Statement	18
A Appendix: Exactness and Limitation	18
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	18
B.1 Preliminaries And Notations	18
B.2 Differentiable Rendering with Projective Exactness	19
B.3 Ray Integral of Isotropic 3D Gaussian	20
C Appendix: Cookbook of Gradient Computation	21
C.1 Chain Rule	21
C.2 Two-Stage Propagation	21
C.3 Gradients for Canonical Ray	21
C.4 Gradients for Whitening Matrix	22
C.5 Gradients for Scaling and Rotation	22
C.6 Gradients for 3D Mean	22
C.7 Numerical Stability of Our Differentiable Framework	22

864	D Appendix: Additional Details in Association	23
865	D.1 Particle Bounding Frustum (PBF) Solver	23
866	D.2 Details in Computing CSF-PBF Intersection	24
867	D.3 Relation to Prior Works	25
868		
869	E Appendix: BEAP Image Inverse Interpolation	25
870	Pinhole Camera Model	26
871	KB Fisheye Model	26
872		
873	F Appendix: High-level Algorithmic Summary	26
874		
875	G Appendix: Additional Implementation Details	27
876	Baselines	27
877	Implementation Details	27
878	Central and peripheral regions in ScanNet++ evaluation	28
879	Cross-camera generalization evaluation on ZipNeRF	28
880		
881	H Appendix: EWA Splatting Preliminaries	28
882	From camera coordinates to ray coordinates	28
883	Local affine approximation	28
884	Splatting approximation error	29
885		
886	I Appendix: Advantage over Prior Ray-Tracing Methods	29
887		
888	J Appendix: Results on Egocentric Dataset	29
889		
890	K Appendix: Additional Material and Results	31
891	K.1 Additional Material.	31
892	K.2 Additional Results.	31
893		
894		
895		
896		
897		
898		
899		
900		
901		
902		
903		
904		
905		
906		
907		
908		
909		
910		
911		
912		
913		
914		
915		
916		
917		

LLM USAGE STATEMENT

We used a large language model (ChatGPT, OpenAI) as a writing assistant. Specifically, the LLM was employed to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic edits to figure captions and section transitions, and (iii) provide alternative phrasings for technical descriptions without altering the scientific content.

The LLM was not involved in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors. The authors take full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

Here, *exactness* means that 3DGEER removes the linear projective approximations used during both “rendering” and “ray-particle association.” The scaling study in Sec. 4.1 shows why this distinction matters empirically: under our settings, the associated errors do not disappear merely by adding more Gaussians. We do not claim exactness in the full physical volumetric sense; ordering, self-attenuation, and overlapping remain approximations and therefore define the main limitations of the framework.

Limitations. Our formulation integrates Gaussians independently, so it does not explicitly solve self-occlusion, and ordering is handled by depth-based sorting. The practical effect is limited but visible in rare cases as occasional popping artifacts, as shown in the supplementary videos. EVER (Mai et al. 2025) is, to our knowledge, the only compared method that analytically resolves ordering and self-occlusion by approximating each Gaussian as a constant-density ellipsoid, but that design pays a substantial runtime cost.

Importantly, prior ray-tracing Gaussian renderers, including Fuzzy Metaballs (Keselman & Hebert 2022) and later 3DGR/UT methods (Moënné-Loccoz et al. 2024; Wu et al. 2025), also rely on the “maximum-response” assumption for ray-Gaussian interaction. This assumption ignores ordering and self-attenuation, yet has been shown to introduce negligible error in practice (see Celarek et al. 2025) Fig. 9-11 / Sec. 7.1) regardless of the number of Gaussians. Thus, our depth-based ordering preserves the practical accuracy level of existing ray-tracing Gaussian renderers while improving the projective-geometry comparison targeted in this work.

Future work. Although ordering and self-occlusion have small measured impact in current Gaussian ray tracing, they remain the gap between projective exactness and full volumetric correctness. Future work could add analytic or learnable self-attenuation models, potentially drawing on ellipsoidal integration ideas from EVER (Mai et al. 2025), while preserving the runtime profile reported here. A more principled ordering model could also reduce the rare popping artifacts seen in extreme configurations. These limitations are orthogonal to the projective-exactness gains measured in our experiments, and addressing them should further improve robustness on large-scale or highly dynamic scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We follow the classic parameterization of 3D Gaussian primitives as introduced in prior work (Zwicker et al. 2001). Each 3D Gaussian distribution is defined by a mean position $\mu \in \mathbb{R}^3$ and a 3D covariance matrix Σ in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where we adopt the normalization constant ρ as $\sqrt{2\pi}$, and $\mathbf{x} \in \mathbb{R}^3$ denotes a point in world space. This formulation differs from recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which omit the determinant term $|\Sigma|^{1/2}$ to simplify splatting or association during

rendering. The form of our covariance matrix obeys: $\Sigma = RSS^T R^T$, given a scaling matrix $S = \text{diag}(\mathbf{s})$ constructed from the scaling vector $\mathbf{s} \in \mathbb{R}^3$, and a rotation matrix $R \in \text{SO}(3)$ derived from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

To evaluate the transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, we have:

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient associated with the Gaussian.

Finally, considering a set of 3D Gaussian particles sorted by depth contributing to the rendering color of the given ray, the rendered color C is computed as:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ derives the view-dependent color of the i -th Gaussian as seen from the optical center $\mathbf{o}$, parameterized using spherical harmonics (SH) (Kerbl et al., 2023). Through minimizing the reconstruction loss between the rendered color and the observed color along each ray, the parameters $\mathbf{s}, \mathbf{q}, \mu$, the coefficient σ as well as the SH coefficients, are jointly optimized via backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The appendix derivation makes the rendering comparison precise by showing how a projective-exact ray-marching formulation can still be evaluated in closed form. The key step is to transform each ray relative to each Gaussian into a canonical coordinate system where the transmittance integral in Eq. B.2 is preserved and the Gaussian becomes isotropic. From this shared space, the exact ray color follows from first principles.

Specifically, we define a canonical coordinate system shared across all 3D Gaussian primitives. A point $\mathbf{u} \in \mathbb{R}^3$ in this canonical space is mapped to world coordinates for each Gaussian via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

Under this transformation, each infinitesimal canonical ray segment $\Delta \mathbf{u}$ corresponds to the world-frame segment $\Delta \mathbf{x} = RS\Delta \mathbf{u}$, so the two coordinate systems can be compared segment by segment.

To ensure that the transmittance contributed by each Gaussian along the ray is preserved across coordinate systems, we require that the elementary transmittance (i.e., the product of density and segment length) remains invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint implies that the Jacobian determinant is absorbed into the measure. Note that $\left| \frac{\Delta \mathbf{x}}{\Delta \mathbf{u}} \right| = |RS| = |\Sigma|^{1/2}$. We can thus express the corresponding canonical density as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp \left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u} \right), \quad (\text{B.6})$$

which is isotropic and shared across all primitives in canonical space.

This gives the central result used in the experiments: the transmittance of each Gaussian along $\mathbf{r}(t)$ can be computed by accumulating density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ in the canonical isotropic Gaussian. The contribution for $t < 0$ is negligible except when the optical center lies within 3-sigma, in which case 3D Gaussian rendering frameworks cull the Gaussian during optimization.

This admits a *closed-form* solution in terms of the perpendicular distance from the ray to the Gaussian center, as captured in Theorem 1

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp \left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 \right), \quad (\text{B.7})$$

where $D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d})$ denotes the perpendicular distance from the Gaussian center (i.e., the origin in the canonical space) to the transformed ray $\mathbf{r}_u(t)$. Intuitively, this quantity also represents the minimal *Mahalanobis distance* from any points on the ray to the Gaussian in world space, measuring how close the ray passes by the particle distribution.

The remaining question is just about how to compute the perpendicular distance from the canonical origin to the transformed ray. We derive the transformed ray's direction vector $\mathbf{d}_u$ and moment vector $\mathbf{m}_u$ as:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1}R^\top) (\mathbf{o} - \boldsymbol{\mu})$ denotes the optical center in canonical space. Based on this, the square of $D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d})$ can be efficiently calculated as:

$$(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

By substituting this distance square into Eq. B.7 and then the derived transmittance value into Eq. B.3 we compute the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *For the canonical comparison case of a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral admits a closed form proportional to*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first translate the coordinate system so that the Gaussian $\mathcal{G}$ is centered at the origin, translating the ray $\mathbf{r}_u$ by the same amount. We then rotate the scene so the ray direction aligns with the z -axis. Because the integral is translation-invariant and the standard Gaussian is isotropic, these transformations preserve the integral value while simplifying the calculation.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Thus, the integral is proportional to $\exp\left(-\frac{1}{2}D^2\right)$, as stated in theorem. $\square$

Note that the density function shared by our 3D primitives in the canonical space (see Eq. B.6) follows the same formulation as in the isotropic case. Given the world-coordinate ray $(\mathbf{o}, \mathbf{d})$ as the input of D , its value also depends on the world-space Gaussian parameters $\boldsymbol{\mu}, \Sigma$, which are used to transform the ray into the canonical coordinate system. Therefore, for each Gaussian contributing to the ray, we derive the transmittance as:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 when picking $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ is the ray-Gaussian transmittance (Eq. B.7), where σ is the opacity coefficient, and α is the ray-Gaussian integral. We denote $\kappa = D_{\mu, \Sigma}^2$ as the minimal squared Mahalanobis distance from the ray to the Gaussian distribution, S as the Gaussian scaling, and R as the rotation matrix in the world space. Additionally, given $\Sigma = RSS^\top R^\top$ as the 3D covariance matrix of the Gaussian in the world space, we have W_{pca} as the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad \text{where } S^{-1} = \text{diag}\left(\frac{1}{s}\right). \quad (\text{C.1})$$

As illustrated in Fig. 2 we can apply the chain rule to find derivatives *w.r.t.* the Gaussian parameters, e.g, scaling vector s , rotation quaternion q , and mean μ :

$$\frac{\partial T}{\partial s} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial s}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial q} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial q}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}, \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

To keep the backward pass comparable in cost to efficient splatting pipelines, we follow the two-stage 3DGS (Kerbl et al. 2023) structure with `RenderCUDA` (RC) and `PreprocessCUDA` (PC). RC performs ray-wise accumulation accelerated by PBF association (Sec. 3.2), while PC operates with Gaussian-wise complexity and uses view-frustum culling for additional speed.

According to Eq. B.7 we can respectively write the gradient *w.r.t.* opacity coefficient and the shared partial derivative for Gaussian parameters in RC as:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients to the Gaussian parameters can then be back-propagated from the squared distance, e.g., $\frac{\partial \kappa}{\partial s}, \frac{\partial \kappa}{\partial q}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We backpropagate to the canonical ray quantities that determine the measured ray-Gaussian distance: origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$, defined in Eq. B.8. These terms directly control the Mahalanobis distance in Eq. B.9 and therefore the gradients that drive reconstruction quality.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = -\left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ is the world-space optical center and $\mathbf{d}$ is the ray direction. After obtaining gradients with respect to the canonical ray parameters, we propagate them to the PCA whitening matrix W_{pca} through outer products with the corresponding world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \frac{\partial \kappa}{\partial W_{\text{pca}}} &= \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{RC}} + \frac{\partial \kappa}{\partial W_{\text{pca}}} \Big|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here, the gradient backpropagated through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, while the gradient through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`. These two components are then summed to yield the complete gradient w.r.t. the PCA transformation matrix W_{pca} .

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\frac{\partial W_{\text{pca}}^{i,j}}{\partial s_k} \Big|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

To derive gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we can write the whitening matrix as:

$$W_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1}(\frac{1}{2} - q_j^2 - q_k^2) & \frac{1}{s_1}(q_i q_j + q_r q_k) & \frac{1}{s_1}(q_i q_k - q_r q_j) \\ \frac{1}{s_2}(q_i q_j - q_r q_k) & \frac{1}{s_2}(\frac{1}{2} - q_i^2 - q_k^2) & \frac{1}{s_2}(q_j q_k + q_r q_i) \\ \frac{1}{s_3}(q_i q_k + q_r q_j) & \frac{1}{s_3}(q_j q_k - q_r q_i) & \frac{1}{s_3}(\frac{1}{2} - q_i^2 - q_j^2) \end{pmatrix}. \quad (\text{C.11})$$

As a result, we derive the following end gradients for the quaternion $\mathbf{q}$ as:

$$\begin{aligned} \frac{\partial W_{\text{pca}}}{\partial q_r} \Big|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \frac{\partial W_{\text{pca}}}{\partial q_i} \Big|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_2} & \frac{q_k}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \\ -\frac{2q_j}{s_3} & \frac{q_r}{s_3} & \frac{q_i}{s_3} \end{pmatrix}, \\ \frac{\partial W_{\text{pca}}}{\partial q_j} \Big|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_j}{s_3} \end{pmatrix}, \quad \frac{\partial W_{\text{pca}}}{\partial q_k} \Big|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_i}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

C.6 GRADIENTS FOR 3D MEAN

The final gradient needed for optimization is the derivative with respect to the Gaussian 3D mean $\boldsymbol{\mu}$:

$$\frac{\partial \kappa}{\partial \boldsymbol{\mu}} \Big|_{\text{PC}} = -W_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

Numerical stability is another practical comparison point because the forward expression, backward gradient chain, and cached intermediate variables directly affect training quality. Prior approaches such as GOF (Yu et al. 2024b) experience unstable backward propagation and therefore add 3D filtering heuristics. Our closed-form expression and exact gradient chain avoid this instability, so the same training objective can be optimized without auxiliary filtering.



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) GOF produces artifacts when degenerate Gaussians are not filtered; (Right) substituting our forward and backward formulations removes the artifacts under the same condition.

The difference can be traced to how prior methods expand $\mathbf{m}_u^\top \mathbf{m}_u$. In exact arithmetic, the following forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

In finite precision, the equivalence breaks down for nearly degenerate Gaussians. When one axis is scaled close to zero, $\mathbf{o}_u$ and $\mathbf{d}_u$ in canonical space become nearly parallel: $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$, while both $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ grow. The subtraction $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ is therefore prone to overflow and cancellation, sometimes producing negative values and unstable backward gradients.

Fig. C.1 (Left) shows the practical consequence for GOF: without additional 3D filtering to remove degenerate Gaussians, severe artifacts appear. The snowflake noise is caused by overflow in Eq. C.14; negative values can be returned too early, while extremely small positive values amplify Gaussian intensity. Because the EWA Jacobian-and-conic approximation remains stable in this case, the artifact pattern is attributable to inaccurate maximum-response computation, including the observed gridlines.

Our CUDA rasterizer instead computes the cross product and its gradients directly (Eq. B.9 for the forward pass and Eqs. C.6, C.9 for the backward pass), avoiding the unstable subtraction. The resulting forward pass is artifact-free and the backward pass provides stable gradients. Fig. C.1 (Right) applies our forward and backward formulations within GOF and removes the instability artifacts even without filtering degenerate flat or needle-like Gaussians.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 provides our detailed derivation. Section D.2 provides implementation considerations required for stable PBF under large-FoV and omnidirectional cameras. Section D.3 clarifies the derivational relationships and explicitly states the conditions under which prior formulas coincide with ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Recall Eq. 8 we define $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ as the plane transformation matrix from the view space to the canonical space. Take the unknown bounds for $c = \tan \theta_{1,2}$ as example, we have:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes i -th row of the matrix, expressed as a 4×1 column vector, and P_{ij} further denotes j -th element. Recall Eq. 9 assuming we are computing the bounding box of the 1-standard-deviation contour ellipsoid of the Gaussian, i.e., $\lambda = 1$, then we have the following quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. Here we denote the equation as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is a 3×3 upper triangular matrix defined with $\mathcal{T}_{ij} = (\mathbf{f} \circ \mathbf{P}_i^\top) \cdot \mathbf{P}_j$.

Similarly, for the bounds of ϕ , we have $c = \tan \phi_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Interestingly, given a camera-view configuration $[R_c \mid \mathbf{t}_c]$, we find that the parameter matrix $\mathcal{T}$ can be efficiently computed from the Gaussian's view-space covariance $\Sigma_c = R_c R S S^\top R^\top R_c^\top$ and 3D mean $\mu_c = R_c \mu + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

Thus, we can compute the angular bounds (defined by their tangent-space center and extent) from:

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

If either quadratic has a negative discriminant, the camera's optical center lies inside the Gaussian ellipsoid and the angular bounds are undefined. We handle this rare case conservatively by clamping the frustum angles to the camera's maximum field of view (FoV), preserving valid association coverage.

This result is computed directly from the true 3D view-space covariance, unlike EWA and UT association, which first approximate a screen-space conic. The comparison therefore isolates the advantage of PBF: tighter bounds without giving up exact projective geometry.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We uniformly sample spherical angles within the maximum field of view (FoV). The resulting x , y , and z coordinates—i.e., the ray direction vectors $\mathbf{d}$ —are projected onto a unit sphere and then transformed into pixel space for inverse color interpolation (see Sec. E). These sampled rays are then grouped according to their corresponding CSFs for subsequent Gaussian association.

For limited-FoV pinhole cameras, PBF angular bounds usually lie inside $[-\frac{\pi}{2}, \frac{\pi}{2}]$, where $\tan$ is monotonic. In that common regime, PBF-CSF intersections are computed by directly comparing tangent values of the angular bounds. Wide-FoV cameras create the difficult case: even if the incidence angle of the Gaussian center remains inside $[-\frac{\pi}{2}, \frac{\pi}{2}]$, the PBF bounds can leave the monotonic region of $\tan$. Direct tangent comparison can then return incorrect intersections, such as matching bounds as $[\theta'_2, \theta_1]$ (Fig. D.1).

To keep the association comparison correct in this regime, we apply a mirror transformation

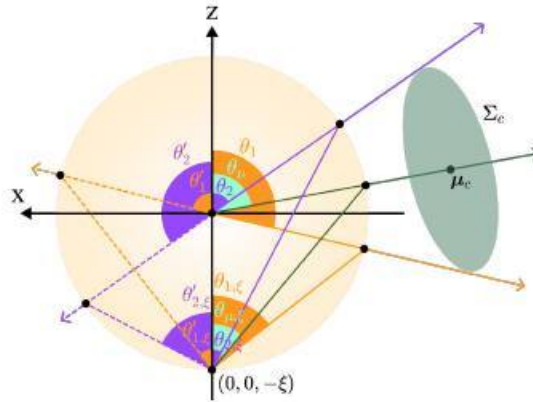


Figure D.1: Mirror Transformation of PBF Angular Bounds. A pair of bounds generates four candidate mirrored angles $\theta_{1,\xi}, \theta'_{1,\xi}, \theta_{2,\xi}, \theta'_{2,\xi}$. For a Gaussian centered at μ_c , the two candidates closest to the mirrored center angle $\theta_{\mu,\xi}$ define robust PBF bounds for comparison with mirror-transformed CSF partitions.

to the tangent values of both PBF bounds and CSF partitions. Following the CMEI camera model (Mei & Rives 2007), we shift the optical center with mirror parameter ξ , giving:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ represents the mirror-transformed angle of θ on the xz -plane, using a mirror parameter ξ . We set $\xi = 1$ throughout all experiments.

For a tangent pair $[\tan \theta_1, \tan \theta_2]$ from the PBF solver (Eq. D.6), the transformation produces four candidate mirror angles (Fig. D.1). The Gaussian center has a unique mirror-transformed angle $\theta_{\mu, \xi}$, so we select the two candidates nearest to $\theta_{\mu, \xi}$. These define the effective PBF lower and upper bounds, $[\theta_{1, \xi}, \theta_{2, \xi}]$, which are then compared with CSF partitions to compute the correct intersections.

This transformation keeps angular comparisons robust beyond the monotonic domain of $\tan$. The timing analysis in Tab. K.5 shows that the full preprocessing path, including mirror-angle PBF-CSF intersection and canonical transformation for all Gaussians, takes less than 0.13 ms. Thus the correction improves wide-FoV association without changing the favorable runtime profile.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen-space coordinates

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation yields the same expressions as 2DGS (Huang et al., 2024a) and HTGS (Hahlbohm et al., 2025). This identifies those perspective-only formulas as a special case of the angular-domain formulation used in 3DGEER.

Concretely, 2DGS and HTGS begin with a screen-space or tangent-plane parameterization, so their derivations rely on perspective-camera identities in which rays are tied to a planar, z -parallel image plane. These assumptions do not extend directly to fisheye or omnidirectional cameras, where the corresponding screen-space surface is curved. 3DGEER instead starts from a camera-agnostic 3D angular parameterization and directly parameterizes CSFs and PBFs. The different starting point changes which quantities are known and which constraints are valid, allowing closed-form bounds and culling guarantees to hold for generic cameras.

Thus, the contribution is not merely algebraic equivalence in the perspective case; it is deriving the bounds under weaker assumptions, without relying on screen-space structure, and validating the resulting method across both pinhole and wide-FoV camera families.

The parameterization also changes the measured computational pipeline because association, duplication, and sorting depend on how Gaussians map to tiles or bins. An angular CSF domain produces a different bucketing structure from pinhole screen-space tiling. This matters for large-FoV scenes: under generic cameras, a Gaussian bound can appear partly “behind” the camera in screen coordinates (Fig. D.1), causing screen-space methods to form mirrored or incorrect tangent bounds (dashed line in Fig. D.1). We therefore introduce mirror parameters in association (Sec. D.2) and use the Gaussian center for sorting and correction, components that are unnecessary in perspective-only derivations but important for the reported wide-FoV results.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

For training, we sample rays so that the original image FoV is covered uniformly in the BEAP representation and feed those rays to the CUDA renderer. For each sampled ray, we interpolate the corresponding ground-truth color, ensuring that the supervision used in the reported comparisons is both accurate and uniformly distributed.

To project sampled rays back to observed images, we first convert the incidence-angle coordinates θ and ϕ into the normalized ray direction $\mathbf{d} = (x, y, z)^\top$:

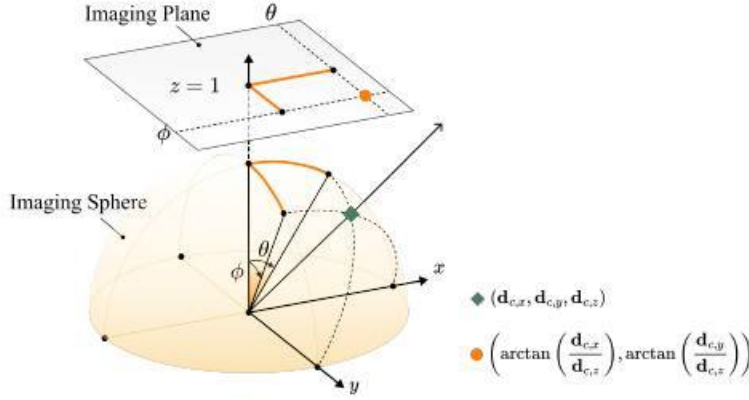


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is normalized onto the optical sphere (green spade) and projected angularly onto the xz and yz planes. The orange point gives the (θ, ϕ) coordinate used by both PBF association and BEAP supervision, aligning the geometric and empirical comparisons.

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

The normalized ray $(x, y, z)^\top$ is then mapped to pixel coordinates $(x_p, y_p, 1)^\top$ using the corresponding camera model, keeping evaluation in the native image domain.

Pinhole Camera Model. For a pinhole camera, the angular coordinates satisfy $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$, yielding the standard intrinsic projection:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. For the KB fisheye model, the projected angular coordinates first determine the incidence angle φ :

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

The model then applies the fisheye distortion polynomial:

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

This distorted angle maps the BEAP-sampled ray into pixel space for inverse color interpolation:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

Alg. 1 and Alg. 2 summarize the full rendering and training pipeline in a format comparable to 3DGS Kerbl et al. (2023). Modules introduced or replaced by 3DGEER are highlighted in green, and modules with substantial changes are marked in yellow, making the implementation differences visible alongside the empirical comparisons.

ALGORITHM 1: GPU Software Asso. & Render.

Function 3DGEER ($\mu, s, q, c, \sigma, r_c, V$) :

```
CullGaussian( $\mu, V$ ) ▷ Frustum Culling
 $F \leftarrow \text{InitCSFs}(w, h, K)$  ▷ Partition
 $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(s, q, V)$  ▷ Transform
 $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$  ▷ Sub-frustum Asso.
 $W_{pca} \leftarrow \text{PCA\_Whitening}(s, q)$  ▷ Transform
 $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$  ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
SortByKeys( $\mathcal{G}_{ID}, F_K$ ) ▷ Globally Depth Sort
 $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
 $I \leftarrow 0$  ▷ Init Canvas (w/ Rays)
for all CSFs  $f \leftarrow I$  do
  for all Rays  $i \leftarrow f$  do
     $r \leftarrow \text{GetFrustumRange}(R, f)$ 
     $m_u, d_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$  ▷ Transform
     $D_{\mu, \Sigma} \leftarrow \text{Dist}(m_u, d_u)$  ▷ Mah-distance
     $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$  ▷ Alpha Blending
  end
end
return  $I$ 
```

ALGORITHM 2: 3DGEER Initialization and Training Framework

 w, h : width and height of the training images K, V : intrinsics and extrinsic meta info.

```
 $\mu \leftarrow \text{SfM Points}$  ▷ Init Gaussian Attributes
 $s, q \leftarrow \text{InitAttributes}()$  ▷ Position
 $c, \sigma \leftarrow \text{InitCoefficients}()$  ▷ Scaling, Rotation
▷ Color, Opacity
while not converged do
   $r_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$  ▷ View Rays / Colors
   $C \leftarrow \text{3DGEER}(\mu, s, q, c, \sigma, r_c, V)$  ▷ Asso. & Render
   $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$  ▷ See Alg. 1
   $\mu, s, q, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$  ▷ Loss
  if IsRefinementIteration( $i$ ) then ▷ Backprop & Step
     $\mu, s, q, c, \sigma \leftarrow \text{Optimize}(\mu, s, q, c, \sigma)$  ▷ Kerbl et al. (2023)
  end
end
```

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. The baseline set is chosen to make each comparison axis explicit. For large-FoV fish-eye inputs, we compare with FisheyeGS (Liao et al. 2024) (fast splatting), EVER (Mai et al. 2025) (exact ellipsoidal ray integration), and 3DGUT (Wu et al. 2025) (ray rendering with approximate association). For pinhole settings, we additionally include 3DGS (Kerbl et al. 2023) and 3DGRT (Moënné-Loccoz et al. 2024).

Implementation Details. To keep the quantitative comparisons fair, ScanNet++ and MipNeRF360 use initialization and hyperparameters aligned with the original 3DGS, while ZipNeRF uses the training configurations of EVER and SMERF (Duckworth et al. 2024) for all candidate methods. As in most ray-based frameworks, 3DGEER does not use mean2D gradients for densification or pruning; it instead optimizes from view-space mean3D gradients. Unless otherwise stated, models share supervision strategies and architectural settings. For fisheye datasets (ScanNet++ and ZipN-

eRF), we precompute bijective grid mappings (Guo et al. 2025) and reproject rendered outputs into the native KB-Fisheye image space (Kannala & Brandt 2006) before measuring metrics. ScanNet++ and MipNeRF360 use full-resolution images; for ZipNeRF, because of the high frame count and extremely large fisheye resolution, we use the maximum resolution in Tab. 3 that lets all baselines run reliably on a local desktop setup.

Central and peripheral regions in ScanNet++ evaluation. To make regional metrics comparable across methods, the central region is defined by rays in the angular range

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}].$$

The peripheral region contains the remaining sampled rays. Here w, h denote image width and height, and f_x, f_y are the focal lengths from the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF contains fisheye raw images and corresponding undistorted frames obtained by distortion correction and central cropping. Although frames are aligned pairwise, COLMAP (Schönbberger & Frahm 2016) independently reconstructs camera poses and point clouds for each set, so a single rigid alignment between the pose sets can introduce additional SfM error. To ensure that cross-camera metrics reflect camera-model generalization rather than pose mismatch, we use the same test-time camera poses as training and vary only the intrinsic parameters while keeping extrinsics fixed.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. As the coordinate definitions in EWA Splatting (Zwicker et al. 2001), we have the following mapping function from the camera (view) space points $\mathbf{t}$ to the ray space points $\mathbf{x}$ for projective transformation in pinhole camera models, *i.e.*, $\tau: \mathbf{t} \rightarrow \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$\text{, where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are not affine, which requires computationally expensive (Westover, 1990; Mao et al. 1995) or locally approximated solutions (Zwicker et al. 2001; Kerbl et al. 2023) to calculate integral in the ray space.

Local affine approximation. EWA splatting by (Zwicker et al. 2001) introduced local affine approximation, denoted as τ_k , to keep the linear effects and the k -indexed reconstruction kernels as Gaussians in 2D screen space. It is defined by the first two terms of the Taylor expansion of τ at the camera space point $\mathbf{t}_k$, then the ray space point $\mathbf{x}$ near around $\mathbf{x}_k$ can be estimated by:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian matrix given by the partial derivatives of τ at the point $\mathbf{t}_k$. For the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a certain camera coordinate, using EWA local affine transformation, the approximated density function can be expressed as a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in the ray coordinate. For $\forall \mathbf{x}$ around $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

Integrating a 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a 2D Gaussian $\mathcal{G}_{\Sigma_k, \hat{\mathbf{x}}_k}^{(2)}$, hence:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\Sigma_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** The visual comparison complements the MipNeRF360 metrics: 3DGEER improves detail relative to both 3DGS-based splatting and 3DPRT-based ray tracing while retaining high runtime efficiency. Colored boxes highlight the largest differences.

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$, and the mean $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 variance matrix $\hat{\Sigma}_k$ is easily obtained from the 3×3 matrix Σ_k by skipping the third row and column.

Splatting approximation error. Although we can not expect the density function in ray space still obey the Gaussian distribution when we apply general projective transformation to the original view space, exact density can be derived through substitution via Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k) \right), \end{aligned} \quad (\text{H.8})$$

which means that the approximation error can be estimated through Monte Carlo integration of the following equation:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Note that x_2 represents the dimension along the ray direction in the ray coordinate.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray tracing, such as 3DGRT, can be projective-exact in principle, but practical acceleration changes the measured outcome. 3DGRT uses inscribed proxy icosahedra for BVH intersection tests; these proxies are tight at vertices but their face centers lie $\sim 20\%$ inward by icosahedron geometry. The proxy can reduce intersection accuracy, degrade accumulated gradients during density control, and lead to incorrect geometry such as the missing leaves in Fig. I.1. Far, tiny, or cluttered Gaussians also stress floating-point precision and can produce noisy renderings. This helps explain why 3DGUT, despite UT approximation, outperforms 3DGRT on MipNeRF360. For a controlled validation, we render the same pretrained MipNeRF360-stump field using the 3DGRT ray-tracing engine and the 3DGEER pipeline. On PSNR $\uparrow$ /SSIM $\uparrow$ /LPIPS $\downarrow$, 3DGRT reports 25.79/0.745/0.264, whereas 3DGEER reports 26.83/0.774/0.246, a margin of +1.04/+0.029/-0.018. Together with BEAP supervision, the full 3DGEER pipeline is both faster and more accurate in Tab. 4.

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Egocentric cameras provide a useful intermediate test case: their FoVs are wide enough to expose projective effects, but not as extreme as the 180° datasets. Existing pipelines often lose quality either on full-resolution wide-FoV images or after cropping, and low-light capture further lowers the signal available to small sensors and aperture-constrained lenses.

On Aria ($\sim 110^\circ$ circular FoV), 3DGEER achieves 33.5dB average PSNR compared with 32.8dB for FisheyeGS and is better across the reported metrics (Tab. J.1). The qualitative results follow the numbers: indoor peripheral floor details are sharper (Fig. J.1 Left), and outdoor distant structures such as trees are clearer (Fig. J.1 Right). The +0.7dB PSNR margin falls between the smaller gains



Figure J.1: **Qualitative Results on the Aria Dataset.** The moderate-FoV egocentric comparison shows sharper indoor floor texture and clearer outdoor tree detail for 3DGEER.

Table J.1: **Quantitative Comparison on Aria.** 3DGEER and FisheyeGS are compared on the same indoor and outdoor egocentric scenes; higher PSNR/SSIM and lower LPIPS indicate better quality.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

on pinhole datasets (e.g., MipNeRF360: +0.3dB; ZipNeRF-cross: +2.0dB) and the larger gains on highly distorted 180° diagonal-FoV datasets (e.g., ScanNet++/ZipNeRF: +0.9–4.8dB), matching Aria’s moderate distortion level.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6**. At extreme FoV, 3DGS collapses to black because $\tan 90^\circ \rightarrow \infty$, whereas 3DGEER robustly renders views up to 180° and reconstructs the green chair where splat-based methods such as FisheyeGS fail. The zoom-ins in Fig. 6 show the same pattern for structures such as corridor/fridge, table/door, and window/cabinet regions.



Figure K.2: **Extreme-FoV Generalization.** (Left) 3DGEER and 3DGS are compared under extreme-FoV pinhole (170°) views after training on undistorted central regions. (Right) Zoom-out results from Fig. 6 show the complete circular 180° FoV, where 3DGEER preserves out-of-distribution regions more robustly.

Table K.1: **MipNeRF360 Quantitative Results by Sequence.** Per-sequence PSNR/SSIM/LPIPS values support the aggregate comparison reported in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

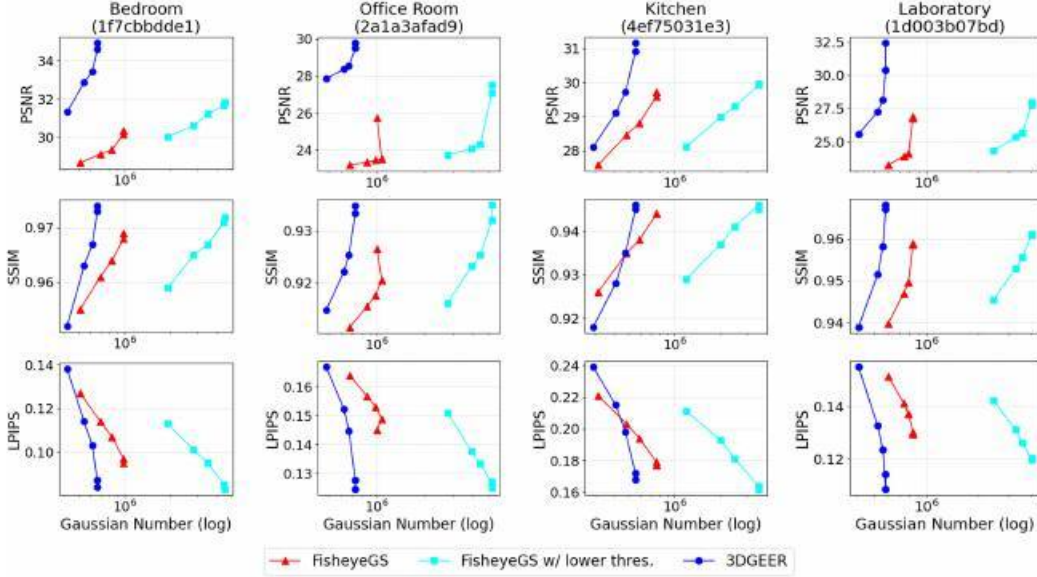


Figure K.3: **PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts.** Full metric trends confirm that increasing the splatting Gaussian count does not close the projective-exactness gap to 3DGEER.

Table K.2: **ScanNet++ Quantitative Results by Sequence (Full FoV).** The overall results are shown in Tab. 2.

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114



Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** Zoomed colored boxes highlight the regions where the quantitative margins appear visually: 3DGEER reconstructs peripheral content better than FisheyeGS, shows fewer artifacts and sharper detail than EVER, and preserves crisper boundaries and textures such as wires, keyboards, and wooden floors compared with 3DGUT.

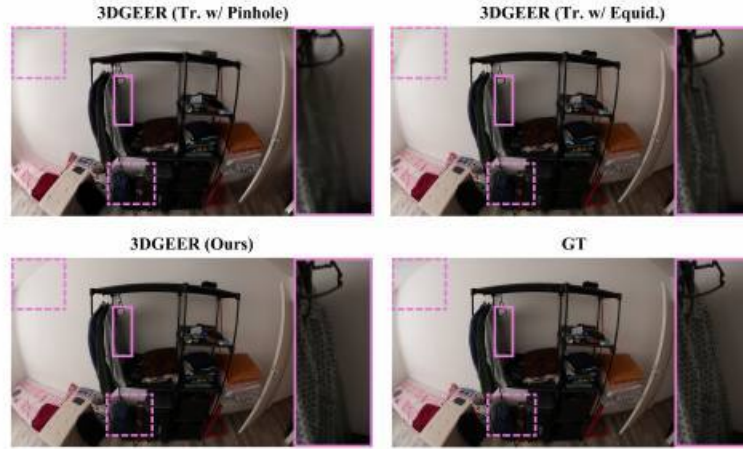


Figure K.5: **Impact of BEAP Supervision.** With the renderer fixed, BEAP’s balanced angular sampling preserves geometric and texture details better than pinhole or equidistant supervision, matching the metric gains in Tab. [K.8](#).

Table K.3: **Quantitative Results on the ScanNet++**. PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts. Bedroom, Office Room, Kitchen, and Laboratory in order.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Other Methods**. Stage-wise timings compare 3DGEER with efficient splatting baselines on MipNeRF360 and ScanNet++, showing how reduced duplication and sorting offset the cost of ray-based rendering.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Runtime Analysis: Association Method on ScanNet++**. PBF association is compared with splatting-based EWA and UT association, both with and without SnugBox acceleration inside the same 3DGEER pipeline, so stage-wise timing differences isolate the association strategy.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Total	Avg. FPS ↑
		Dup.	Sort	Render			
EWA	0.13	1.17	3.27	12.69	17.84	56	
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115	
UT	0.13	0.71	1.94	7.97	11.16	90	
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228	
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251	

Table K.6: **Impact of PBF-CSF Association**. The artifact-sensitive LPIPS metric is paired with memory use, Gaussians per tile, and cross-method LPIPS to compare association quality. Δ LPIPS measures the gap between in-method and cross-method evaluation, quantifying consistency and generalization under each association strategy.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Training-Time Transmittance Function Replacement**. Per-scene ScanNet++ results decompose the aggregate transmittance comparison in Tab. 5.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Impact of Different Supervision Spaces**. Per-scene ScanNet++ results show how central perspective, full perspective, equidistant, and BEAP supervision compare under the same 3DGEER renderer; aggregate metrics appear in Tab. 6.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **ZipNeRF Quantitative Results by Sequence**. Alameda, Berlin, London, and NYC are reported in order, decomposing the aggregate cross-camera comparison in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220